SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	A. Mounish	**Reg. No.:**	192472273
Section / Batch:	CSE AI	**Date:**	10/08/26

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.

· Implement the solution using **Python**.
· Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
· Use **embeddings and semantic search** where applicable.
· Use **FAISS or ChromaDB** as the vector database for RAG tasks.
· Demonstrate the complete pipeline:

Document → Chunking → Embedding → Retrieval → Generation

· Demonstrate the system with multiple queries.
· Handle irrelevant, incomplete, or unsupported queries appropriately.
· Display retrieved context/source wherever applicable.
· Explain the system architecture.
· Provide a working end-to-end demonstration.

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.* Signature of the Student:
_____

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)
DocuRAG: RAG-Based Software Documentation Assistant

1. Software Documentation Assistant: Create a RAG assistant using software/API documentation and answer technical questions using only the supplied documentation.

1. Question

Create a RAG assistant using software/API documentation and answer technical questions using only the supplied documentation.

2. Aim

To develop a Retrieval-Augmented Generation assistant that can process software/API documentation, retrieve relevant technical information for a user query, and generate an accurate answer using only the supplied documentation as the source of knowledge.

3. Problem Statement

Software and API documentation often contains a large amount of technical information distributed across different sections and pages. Finding the correct information manually can be time-consuming. A documentation assistant can use semantic retrieval to locate relevant documentation and provide a concise answer. However, the assistant must remain grounded in the supplied documentation and should not introduce unsupported information from general model knowledge.

4. Objectives

- Accept software/API documentation through a PDF upload interface.
- Extract and preprocess the supplied documentation.
- Divide the document into searchable text chunks.
- Generate semantic embeddings for the document chunks.
- Store the embeddings in a FAISS vector index.
- Retrieve the most relevant documentation for a technical query.
- Generate answers using only the retrieved documentation.
- Display retrieved sources and document page information.
- Prevent unsupported answers when relevant documentation is unavailable.
- Provide an interactive web-based documentation assistant.

5. System Overview

DocuRAG is implemented as an interactive software documentation question-answering system. The user first uploads the supplied software/API documentation. The system extracts the document text, divides it into overlapping chunks, and converts each chunk into a semantic embedding. FAISS stores the embeddings and performs similarity-based retrieval. When a technical question is entered, the question is embedded using the same embedding model and compared against the indexed documentation. The most relevant contexts are then supplied to the language model. The model is explicitly instructed to answer only from the supplied documentation.

6. Architecture

The complete architecture consists of document ingestion, preprocessing, semantic representation, vector retrieval, grounded generation, and source presentation.

Software/API PDF
↓
Text Extraction
↓
Text Chunking
↓
Sentence-Transformer Embeddings
↓
FAISS Vector Index
↓
Technical User Query
↓
Query Embedding + Similarity Search
↓
Relevant Documentation Context
↓
Gemini Grounded Generation
↓
Technical Answer + Source Pages

7. Technologies Used

Technology / Component	Role	Purpose
Python	Programming language	Implements the complete RAG pipeline.
Streamlit	User interface	Provides an interactive web application.
PyPDF	PDF processing	Extracts text from documentation pages.
Sentence Transformers	Embedding generation	Creates semantic vector representations.
all-MiniLM-L6-v2	Embedding model	Generates 384-dimensional embeddings.
FAISS	Vector retrieval	Performs similarity-based

		document retrieval.
Google Gemini	Language model	Generates grounded technical answers.
VS Code	Development environment	Used to develop and execute the application.

8. Implementation Methodology

Document Upload: The user uploads the software/API documentation in PDF format.

Text Extraction: The PDF is processed page by page and readable text is extracted.

Chunk Creation: The extracted text is divided into overlapping chunks so individual sections can be retrieved.

Embedding Generation: Each chunk is converted into a semantic vector using the all-MiniLM-L6-v2 model.

Vector Indexing: The generated vectors are stored in a FAISS index for efficient similarity search.

Technical Query: The user enters a technical question related to the supplied documentation.

Query Embedding: The question is converted into a vector using the same embedding model.

Semantic Retrieval: FAISS retrieves the most relevant document chunks according to semantic similarity.

Grounded Generation: The retrieved documentation is supplied to Gemini with strict instructions to use only that evidence.

Source Presentation: The application displays the relevant documentation pages and retrieved context.

9. Grounded Question Answering

The most important design decision in the assistant is documentation-only generation. The generation prompt explicitly instructs the language model not to use outside knowledge, internet information, invented APIs, invented parameters, invented commands, or unsupported assumptions. If the supplied documentation does not contain enough information, the system is designed to state that the information could not be found rather than producing an unsupported answer.

10. User Interface and Document Indexing

The completed application provides a dedicated Software Documentation Assistant interface. The sidebar displays the RAG pipeline, including Documentation, Text Chunking, Semantic Embeddings, FAISS Retrieval, and Grounded Generation. The main interface provides software/API document upload and reports successful indexing before the user begins technical question answering.

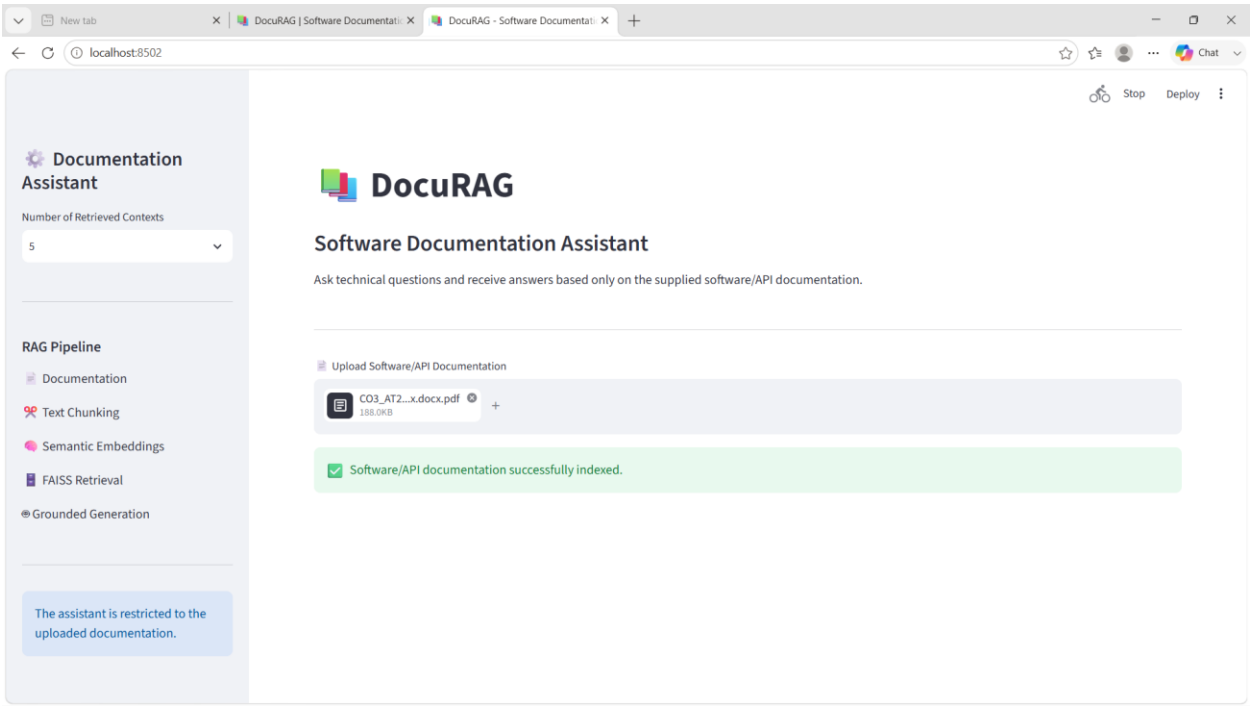

Figure 1: DocuRAG Software Documentation Assistant after successful document indexing.

11. Technical Question Answering Workflow

After indexing, the user selects the required retrieval depth and enters a technical question. The query is converted into an embedding and compared with the document embeddings. The highest-ranked documentation chunks are displayed as retrieved evidence. The language model then generates a technical answer using these retrieved chunks. Source pages are displayed so that the user can verify the answer against the original documentation.

12. Documentation-Only Verification

The assistant was designed to distinguish between information available in the supplied documentation and information that is not supported by the document. During testing, an unrelated question such as 'What is the price of an iPhone 17?' was rejected because the required information was not present in the supplied documentation. This demonstrates that the system does not blindly answer every question using general model knowledge.

13. Functional Results

Function	Expected Behavior	Status
PDF Upload	Accept and process supplied documentation	Successfully completed
Text Extraction	Extract document text page by page	Successfully completed
Chunking	Create searchable text chunks	Successfully completed
Embeddings	Generate semantic representations	Successfully completed
FAISS Retrieval	Retrieve relevant documentation	Successfully completed
Technical QA	Answer from supplied documentation	Successfully completed

Source Display	Show supporting document pages	Successfully completed
Groundedness	Reject unsupported information	Successfully demonstrated

14. Advantages

- Reduces the time required to search technical documentation.
- Uses semantic retrieval rather than relying only on exact keyword matching.
- Provides source-page information for answer verification.
- Restricts generation to supplied documentation.
- Supports different retrieval depths through Top-K selection.
- Provides an interactive and easy-to-demonstrate interface.
- Can be adapted to different software and API documentation sets.

15. Limitations

- The quality of the answer depends on the quality and completeness of the supplied documentation.
- Scanned PDFs may require OCR before text can be retrieved.
- Character-based chunking may sometimes divide related information across chunks.
- Semantic similarity depends on the selected embedding model.
- The generation stage requires a configured Gemini API key.

16. Future Scope

- Support multiple documentation files simultaneously.
- Add hybrid keyword and semantic retrieval.
- Add a reranking model to improve context selection.
- Store document indexes persistently for repeated use.
- Add OCR for scanned documentation.
- Support additional document formats such as DOCX, HTML, and Markdown.
- Add conversation history for multi-turn technical support.
- Add automated retrieval and answer evaluation metrics.

17. Rubric Alignment

Rubric Criterion	**Implementation Evidence**	**Assessment Strength**
Pipeline Functionality (30%)	Document ingestion, embeddings, FAISS retrieval and Gemini generation are integrated.	Complete end-to-end RAG pipeline.
Answer Relevance & Groundedness (25%)	Retrieved documentation is supplied to the LLM and unsupported questions are rejected.	Strong documentation-grounded behavior.
Query Robustness (20%)	The assistant accepts technical questions and performs semantic retrieval.	Handles varied documentation queries.

End-to-End Demo (15%)	Upload → indexing → query → retrieval → answer → source display.	Complete interactive demonstration.
Architecture Explanation (10%)	Architecture, methodology, components and design decisions are documented.	Clear technical explanation.

18. Result

The Software Documentation Assistant was successfully implemented as a RAG-based technical question-answering system. It accepts software/API documentation, creates a searchable semantic representation, retrieves relevant technical evidence, and generates answers using only the supplied documentation. The application also provides source information and a mechanism for rejecting unsupported questions, thereby satisfying the core requirement of the Software Documentation Assistant.

19. Conclusion

The developed DocuRAG assistant demonstrates how Retrieval-Augmented Generation can be applied to software and API documentation. By combining document processing, semantic embeddings, FAISS retrieval, and grounded language-model generation, the system provides a practical technical documentation assistant while reducing unsupported or hallucinated responses. The completed implementation directly addresses Question 2 of the RAG-based mini assignment.